

CHAPTER – VII

PACKAGES:

- A package in Java is a namespace that groups related classes and interfaces to make code organized, reusable, and conflict-free.

SYNTAX:

```
package packageName;  
  
public class className{  
    //class body  
}
```

ADVANTAGES OF PACKAGES:

- Organization
- Reusability
- Name conflict avoidance
- Access control
- Easier maintenance

TYPES:

- Built – in packages : provided by java API.
- User – defined : created by the programmer.

STATIC IMPORT:

- Static import allows you to access static members(variables or methods) of a class directly without using the class name.

SYNTAX:

```
import static packageName.className.staticMember;
```

EXAMPLE:

```
import static java.lang.Math.*;

public class Demo {

    public static void main(String[] args) {

        System.out.println(sqrt(25));

        System.out.println(PI);

    }

}
```

ACCESS MODIFIER IN JAVA:

- Access modifiers controls the visibility (scope) of classes, methods and variables.

TYPES:

- public – Everywhere
- protected – Package + Subclass
- default (no keyword) – Package only
- private – class only

EXAMPLE:

```
class Student {

    private int age = 20;

    void display() {

        System.out.println(age);

    }

}
```

```
}  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

ENCAPSULATION:

- Encapsulation is a process of binding data and methods together into a single unit.
- It restrict direct access to the data from outside.

EXAMPLE:

```
class Student {  
    private String name;  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
Student s = new Student();  
s.setName("Shalini");  
System.out.println("Name: " + s.getName());  
}  
}
```

GETTER:

- Used to get or return the value of private variable.
- Getter methods – to read data

SETTER:

- Used to set or modify the value of a private variable.
- Setter methods – to write data

SYNTAX:

```
class className {  
    private dataType variableName;  
    public dataType getVariableName() {  
        return variableName;  
    }  
    public void setvariableName(dataType value) {  
        variableName= value;  
    }  
}
```

EXAMPLE:

```
class Employee {  
    private String name;  
    public void setName(String name) {  
        this.name = name;  
    }  
    public String getName() {  
        return name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Employee emp = new Employee();  
        emp.setName("Shalini");  
        System.out.println("Employee Name: " + emp.getName());  
    }  
}
```